Ejecución — guía breve (una página)

Requisitos: NetBeans 31, JDK completo 21 o superior (tu JDK 25 es compatible con release 21), Maven 3.9+, MySQL 8.4. Connector/J y BCrypt se descargan con Maven. Confirmar servidor con SELECT VERSION();; “26.7” queda pendiente de identificar.

NetBeans: Archivo → Abrir proyecto → seleccionar la carpeta que contiene pom.xml. Elegir un JDK completo en Herramientas → Plataformas Java y en las propiedades del proyecto. Ejecutar Clean and Build. Abrir src/main/java/com/progra/pos/view/ConsolaApp.java y ejecutar el proyecto usando su main. Si la acción Maven no configura clase principal, usar el objetivo compile exec:java -Dpos.mainClass=com.progra.pos.view.ConsolaApp.

Base de datos local propia: crear una BD vacía pos_dev con utf8mb4; ejecutar sql/01_schema.sql una sola vez desde MySQL Workbench o el cliente mysql. Crear una cuenta de aplicación dedicada. Para uso normal basta SELECT, INSERT, UPDATE y DELETE sobre esa BD. El esquema crea Consumidor Final y no crea contraseñas compartidas.

Conexión: configurar en el proceso que inicia NetBeans POS_DB_URL, POS_DB_USER, POS_DB_PASSWORD y reiniciar el IDE. Ejemplo URL: jdbc:mysql://localhost:3306/pos_dev?sslMode=VERIFY_IDENTITY. Esa opción requiere certificado confiable válido para el hostname; configurar truststore Connector/J si la CA es privada. No guardar valores secretos en Git ni en capturas. Usuario MySQL y usuario del POS son cuentas distintas.

Administrador inicial: en una terminal con las variables configuradas ejecutar mvn compile exec:java -Dpos.mainClass=com.progra.pos.view.ConsolaApp -Dexec.args=--crear-admin; introducir nombre, username y una contraseña propia de al menos 8 caracteres. Solo funciona con usuarios vacío. Después ejecutar mvn compile exec:java -Dpos.mainClass=com.progra.pos.view.ConsolaApp, iniciar sesión y crear un cajero con usuario-crear. Credenciales de prueba: las que acabas de crear; no hay claves fijas en el repositorio.

Alternativa Linux/Docker incluida: desde la raíz ejecutar scripts/mysql-local.sh, luego scripts/run.sh -Dexec.args=--crear-admin y después scripts/run.sh. El script usa MySQL local en puerto 3307, genera claves en .local/mysql.env ignorado por Git y confía en la CA local mediante un truststore. JDK completo, Docker, Python >= 3.12 y curl deben estar disponibles; scripts/mvn.sh descarga Maven si falta. No eliminar el contenedor si necesitas sus datos; detener con docker stop progra-mysql-fase2 y reiniciar con mysql-local.sh. El script preserva el esquema existente y no aplica migraciones.

Pruebas: mvn test ejecuta modelo; las pruebas MySQL se omiten si no habilitas la BD. En Linux/Docker ejecutar scripts/test-mysql.sh para ambas suites contra pos_test, exclusivamente de pruebas: borra sus filas antes de cada caso. No apuntar a datos que deban conservarse. Compilar antes de demo y revisar el resumen de Surefire en target/surefire-reports. El detalle de resultados reales está en validacion.md.

Errores habituales: versión release no soportada → elegir JDK completo, no JRE; conexión rechazada → comprobar servicio/puerto; TLS → configurar certificado/CA; duplicado o FK → revisar código, nombre o referencias; Maven en nube con proxy → scripts/mvn.sh genera configuración local sin publicar credenciales. La consola muestra errores de entradas sin cerrar el menú; el login fallido termina y requiere ejecutar nuevamente.